

# Introduction

These notes explain how to build a complete data platform for a high-growth retail company that operates both physical stores and an online e-commerce channel. The content covers the end-to-end pipeline: ingesting weekly CSV files into HDFS and Hive (raw layer), transforming and validating data using Spark (staging layer), and finally building an analytics (gold) layer using a star schema (fact and dimension tables). The notes also emphasize real-world engineering requirements such as preventing data loss, enabling recovery, handling bad/corrupt records without failing jobs, choosing correct datatypes, and applying performance optimizations like partitioning, bucketing, and columnar file formats.

---

## Company and Data Landscape

Scaler Retail Pvt. Ltd. is a multi-city retail business with:

- **Offline operations:** stores, POS systems, warehouse stock movement, replenishment.
- **Online operations:** website/app browsing, cart events, orders, digital payments.
- **High data intensity:** thousands of daily transactions across channels.
- **Need for modernization:** centralized, scalable platform for real-time insight and reporting across sales, inventory, supply chain, and customer experience.

## Why a Centralized Data Platform is Needed

As the company scales, data grows in:

- **Volume:** many transactions, orders, payments, inventory records.
- **Velocity:** continuous updates from store systems and online activity.
- **Variety:** structured (orders), semi-structured behavior logs (clickstream), operational stock data.

## Business Outcomes Enabled by the Platform

- Centralized reporting (single source of truth)

- Operational monitoring (inventory health, payment failures, cancellations)
- Strategic decisions (top products, customer value, regional performance)
- Faster analytics (partition pruning, columnar storage)

---

## Source Datasets and Their Roles

This project uses weekly-delivered CSV files that represent different business domains.

### Customers Dataset

Captures customer registrations across online/offline channels.

| Column Name  | Data Type | Description                |
|--------------|-----------|----------------------------|
| customer_id  | STRING    | Unique customer identifier |
| first_name   | STRING    | Customer first name        |
| last_name    | STRING    | Customer last name         |
| email        | STRING    | Email address              |
| phone_number | STRING    | Contact number             |
| city         | STRING    | Customer city              |
| state        | STRING    | State                      |

|               |        |                              |
|---------------|--------|------------------------------|
| signup_date   | DATE   | Date of registration         |
| customer_type | STRING | Online / Offline / Both      |
| is_active     | String | If customer is active or not |

**Key Business Use**

- Customer identity and segmentation
- City/state distribution
- Active vs inactive customer population

**Important Columns**

| Column        | Meaning             | Typical Data Quality Risks              |
|---------------|---------------------|-----------------------------------------|
| customer_id   | unique customer key | duplicates, nulls                       |
| email         | customer email      | inconsistent casing, invalid formats    |
| customer_type | Online/Offline/Both | invalid values, spelling variations     |
| is_active     | active flag         | inconsistent values like Y/N/true/false |

**Stores Dataset**

Represents physical stores.

| Column Name | Data Type | Description     |
|-------------|-----------|-----------------|
| store_id    | STRING    | Unique store ID |

|              |        |                             |
|--------------|--------|-----------------------------|
| store_name   | STRING | Store name                  |
| city         | STRING | City                        |
| state        | STRING | State                       |
| region       | STRING | North / South / East / West |
| opening_date | DATE   | Store opening date          |
| store_type   | String | Type of store               |

Key Business Use

- Regional reporting (North/South/East/West)
- Store performance analysis
- Store lifecycle tracking (opening\_date)

Products Dataset

Product master data including category, subcategory, pricing.

| Column Name  | Data Type | Description       |
|--------------|-----------|-------------------|
| product_id   | STRING    | Unique product ID |
| product_name | STRING    | Product name      |

|              |        |                                        |
|--------------|--------|----------------------------------------|
| category     | STRING | Electronics / Fashion / Grocery / Home |
| sub_category | STRING | Sub-category                           |
| brand        | STRING | Brand name                             |
| price        | DOUBLE | Selling price                          |
| cost_price   | DOUBLE | Procurement cost                       |
| launch_date  | DATE   | Product launch date                    |
| is_active    | string |                                        |

**Key Business Use**

- Product dimension for analytics
- Profitability insights (price vs cost\_price)
- Category/brand performance

**Important Columns**

| Column     | Meaning                          | Typical Data Quality Risks   |
|------------|----------------------------------|------------------------------|
| price      | selling price                    | negative values, wrong types |
| cost_price | procurement cost                 | missing, inconsistent        |
| category   | Electronics/Fashion/Grocery/Home | invalid category values      |
| is_active  | active flag                      | inconsistent values          |

# Inventory Dataset

Tracks inventory across stores and warehouses.

| Column Name        | Data Type | Description              |
|--------------------|-----------|--------------------------|
| inventory_id       | STRING    | Unique record ID         |
| product_id         | STRING    | Product reference        |
| location_type      | STRING    | Store / Warehouse        |
| location_id        | STRING    | Store ID or Warehouse ID |
| quantity_available | INT       | Current stock            |
| reorder_level      | INT       |                          |
| last_updated       | TIMESTAMP | Last update time         |

## Key Business Use

- Stock monitoring and replenishment
- Store vs warehouse availability
- Preventing stockouts (reorder\_level)

## Important Columns

| Column             | Meaning     | Typical Data Quality Risks |
|--------------------|-------------|----------------------------|
| quantity_available | stock count | negative values            |

location\_typeStore/Warehouseinconsistent values (store, STORE, wh)

last\_updatedupdate timemalformed timestamps

Orders Dataset (High-Volume Fact)

Represents both online and store transactions. This is the most important and high-volume table.

| Column Name     | Data Type | Description                              |
|-----------------|-----------|------------------------------------------|
| order_id        | STRING    | Unique order ID                          |
| customer_id     | STRING    | Customer reference                       |
| order_channel   | STRING    | Online / Store                           |
| store_id        | STRING    | Nullable (only for store purchases)      |
| order_source    | STRING    |                                          |
| order_date      | DATE      | Date of order                            |
| order_timestamp | TIMESTAMP | Exact time                               |
| order_status    | STRING    | Placed / Shipped / Delivered / Cancelled |
| total_amount    | DOUBLE    | Total order value                        |

|            |        |                   |
|------------|--------|-------------------|
| payment_id | STRING | Payment reference |
|------------|--------|-------------------|

**Key Business Use**

- Sales reporting
- Order lifecycle tracking (Placed → Shipped → Delivered)
- Cancellation and failure tracking
- Partitioning candidate by order\_date

**Important Columns**

| Column        | Meaning                                | Typical Data Quality Risks |
|---------------|----------------------------------------|----------------------------|
| order_channel | Online/Store                           | invalid values             |
| store_id      | null for online                        | wrong non-null store_id    |
| order_status  | Placed/Shipped/Delivered/<br>Cancelled | inconsistent status values |
| total_amount  | total order value                      | negative values            |

**Order Items Dataset**

Line-level items per order.

| Column Name   | Data Type | Description       |
|---------------|-----------|-------------------|
| order_item_id | STRING    | Unique row ID     |
| order_id      | STRING    | Order reference   |
| product_id    | STRING    | Product reference |



|                 |        |                    |
|-----------------|--------|--------------------|
| quantity        | INT    | Quantity purchased |
| item_price      | DOUBLE | Price per unit     |
| discount_amount | DOUBLE | Discount applied   |
| line_total      | Double |                    |

**Key Business Use**

- Product-level sales analytics
- Basket composition (multiple products per order)
- Revenue and discount analytics

**Payments Dataset**

Payment transaction details, linked to orders.

| Column Name    | Data Type | Description                    |
|----------------|-----------|--------------------------------|
| payment_id     | STRING    | Unique payment ID              |
| order_id       | STRING    | Order reference                |
| payment_method | STRING    | UPI / Card / NetBanking / Cash |
| payment_status | STRING    | Success / Failed               |

|                   |           |              |
|-------------------|-----------|--------------|
| failure_reason    | STRING    |              |
| payment_timestamp | TIMESTAMP | Payment time |
| amount_paid       | DOUBLE    | Amount paid  |

**Key Business Use**

- Payment success rates
- Failure analysis (failure\_reason)
- Payment method mix

**Important Columns**

| Column         | Meaning            | Typical Data Quality Risks    |
|----------------|--------------------|-------------------------------|
| payment_status | Success/Failed     | inconsistent values           |
| amount_paid    | amount             | mismatch with order total     |
| failure_reason | reason for failure | nulls for successful payments |

**Clickstream Logs Dataset (Streaming Simulation)**

Tracks website/app behavior such as view, add\_to\_cart, purchase, search.

| Column Name | Data Type |
|-------------|-----------|
| event_id    | STRING    |
| customer_id | STRING    |

|                 |                                                 |
|-----------------|-------------------------------------------------|
| session_id      | STRING                                          |
| event_type      | STRING (view / add_to_cart / purchase / search) |
| product_id      | STRING                                          |
| event_timestamp | TIMESTAMP                                       |
| device_type     | STRING (Mobile / Desktop)                       |
| location_city   | STRING                                          |
| page_url        | string                                          |
| referrer_source | string                                          |

### Key Business Use

- Funnel analysis (views → cart → purchase)
- Conversion by device/city
- Store vs online conversion correlation (if integrated)

### FileSource:

<https://drive.google.com/drive/u/0/folders/1oPBELdKx2gcZ6Uamur45729EXIt7j716>

## Relationships and Data Integrity

Data is linked via foreign keys (FK). These relationships drive joins in staging and analytics.

## Key Relationship Map

| Child Table | FK          | Parent Table | Why it Matters         |
|-------------|-------------|--------------|------------------------|
| orders      | customer_id | customers    | customer-level sales   |
| orders      | store_id    | stores       | store performance      |
| orders      | payment_id  | payments     | payment reconciliation |
| order_items | order_id    | orders       | order line modeling    |
| order_items | product_id  | products     | product analytics      |
| payments    | order_id    | orders       | validation and status  |
| inventory   | product_id  | products     | stock by product       |
| clickstream | customer_id | customers    | behavior by customer   |
| clickstream | product_id  | products     | product interest       |

### Common Integrity Problems to Expect

- Orders referencing missing customers (bad FK)
- Order items referencing missing orders or products
- Payments duplicated or not matching any order
- Clickstream events without customer\_id (anonymous browsing)

---

## Raw Layer Ingestion (HDFS + Hive)

The raw layer stores data as received from source systems. It should be **immutable** and **recoverable**.

### Raw Layer Responsibilities

- Receive weekly CSV files and store them in HDFS in a controlled folder.

- Create Hive database: `{your_login_username}_raw`
- Create Hive external tables for each dataset.
- Load data with **no data loss**.
- Ensure recoverability if tables are dropped.

## Recommended Raw Storage Design

### Use External Tables for Recoverability

- **External table** means:
  - Table metadata is in Hive Metastore.
  - Data files remain in HDFS even if the table is dropped.

### HDFS Folder Structure (Example)

- `/data/{username}/raw/customers/ingest_date=YYYY-MM-DD/`
- `/data/{username}/raw/orders/ingest_date=YYYY-MM-DD/`
- Similar paths for each file.

This supports:

- Traceability by ingest week/date
- Replay of old loads
- Easy rollback

## Preventing Data Loss During Load

Key practices:

- Copy raw CSV into HDFS first, never load directly from local.

- Keep the original weekly file unchanged in an “archive” folder in HDFS.
- Validate row counts before and after loading.
- Use staging HDFS location during ingestion, then move to final path after success.

### Practical Validation Checklist

- Compare record counts:
  - CSV row count (minus header) vs Hive table count
- Check nulls in key columns:
  - customer\_id, product\_id, order\_id, payment\_id
- Detect duplicates in primary identifiers:
  - customer\_id in customers, order\_id in orders, payment\_id in payments

### Correct Datatypes for Hive (Raw)

The given schema types are not always optimal. Hive should store types that support analytics and prevent parsing issues.

### Recommended Hive Types

| Dataset     | Column            | Recommended Hive Type               | Reason                          |
|-------------|-------------------|-------------------------------------|---------------------------------|
| customers   | signup_date       | DATE                                | date functions and partitioning |
| customers   | is_active         | BOOLEAN (or STRING if inconsistent) | boolean analytics               |
| products    | price, cost_price | DECIMAL(12,2)                       | money-safe precision            |
| orders      | total_amount      | DECIMAL(14,2)                       | avoid floating errors           |
| orders      | order_timestamp   | TIMESTAMP                           | time analysis                   |
| order_items | quantity          | INT                                 | numeric operations              |
| order_items | discount_amount   | DECIMAL(12,2)                       | currency-safe                   |

|             |                   |           |                      |
|-------------|-------------------|-----------|----------------------|
| payments    | payment_timestamp | TIMESTAMP | time analysis        |
| inventory   | last_updated      | TIMESTAMP | freshness checks     |
| clickstream | event_timestamp   | TIMESTAMP | partition derivation |

### **When NOT NULL is Reasonable (Even in Raw)**

Raw typically accepts everything, but you can still enforce expectations by:

- Keeping raw as permissive (nullable)
- Enforcing strictness in staging

Recommended keys expected non-null:

- customers.customer\_id
- products.product\_id
- orders.order\_id
- order\_items.order\_item\_id
- payments.payment\_id
- clickstream.event\_id

### **Optimization in Raw Layer**

Raw layer is usually not heavily optimized because it is meant to preserve source.

Still, basic optimizations help:

- Convert CSV to columnar later (staging/analytics)
- Avoid heavy bucketing here unless required

### **Partitioning Candidate in Raw**

- `orders_raw` partition by `order_date` is practical because:
  - very high volume
  - frequent date-based queries
  - natural partition key mentioned in requirements

However, many organizations keep raw unpartitioned and do partitioning in staging. Choose based on:

- query frequency on raw
- ingestion simplicity
- storage management expectations

## Bad Data and Duplicates in Raw

Raw layer strategy options:

### Option A: Dump Everything, Mark Bad Rows Separately (Recommended Enterprise Pattern)

- Store all rows in raw
- Also store rejected rows into:
  - `{username}_raw_rejects.<table>_rejects`
- Advantages:
  - perfect audit trail
  - no loss of source information

### Option B: Only Load Valid Rows Into Raw

Not recommended if you need an immutable record of source delivery.



## Do We Need a Data Model in Raw?

Raw layer is typically:

- **Source-aligned** and **schema-aligned**
- Not modeled
- Not transformed beyond datatype parsing

Modeling belongs to the analytics layer (star schema).

---

## Staging Layer (Spark Transformations: Raw → Staging)

Staging layer stores cleaned, standardized, validated data.

### Why Staging Layer Exists

- Raw is messy and inconsistent.
- Downstream analytics needs consistent types and business values.
- Staging acts as:
  - transformation zone
  - validation checkpoint
  - foundation for curated/gold layers

### Staging Layer Responsibilities

- Create Hive database: `{your_login_username}_staging`
- Read raw Hive tables in Spark (YARN) using Hive-enabled SparkSession
- Clean and validate each dataset
- Discard invalid/corrupt records without failing Spark jobs

- Enrich datasets by joining key dimensions
- Write clean outputs back to Hive staging tables
- Apply partition strategy for performance

## Spark Reading Requirements (Hive Support)

Key requirements:

- Use SparkSession with Hive support enabled
- Read from `{username}_raw.<table>`

Reason:

- ensures Hive Metastore access
- consistent schema inference via Hive tables rather than raw CSV inference

## Data Quality Handling Principle

The job must:

- **not fail due to bad records**
- **discard invalid records**
- **ensure only conforming records reach staging**

## Common Spark Techniques to Avoid Failures

- Explicit schema enforcement (avoid inference errors)
- Use defensive parsing for timestamps/dates
- Filter invalid values early
- Handle nulls for mandatory columns

- Use deduplication using business keys

## Customers Staging Transformations

### Duplicate Removal

- Remove duplicate records based on `customer_id`
- Decide “best record” selection logic, such as:
  - latest signup\_date
  - non-null email preferred
  - latest data if weekly incremental loads exist

### Email Standardization

- Convert email to lowercase
- Trim whitespace

Purpose:

- consistent joins
- consistent customer identity

### Name Cleanup

- Trim whitespace from first\_name and last\_name
- Optionally standardize capitalization if needed (not required but useful)

### Null Phone Handling

Possible approaches:

- Keep null (valid if phone is optional)
- Create derived flag: `has_phone_number`
- Validate length/format if required

## Customer Type Validation

Expected values:

- Online
- Offline
- Both

Actions:

- Standardize casing (online → Online)
- Discard invalid values or map them to “Unknown” based on project rule
- In strict staging, discarding invalid is common

## Orders Staging Transformations

### Remove Negative Total Amount

Rule:

- `total_amount >= 0`

Reason:

- negative revenue breaks analytics
- can indicate corrupt ingestion or refunds not modeled separately

### Filter Invalid Order Status

Allowed:

- Placed
- Shipped
- Delivered
- Cancelled

Action:

- standardize casing
- discard anything outside allowed list

## **Ensure order\_date Not Null**

Reason:

- partitioning requires it
- date analytics depends on it

## **Convert Timestamps Properly**

- Ensure `order_timestamp` is parsed as `TIMESTAMP`
- If timestamp parsing fails:
  - discard record (as per requirement)
  - or route to rejects (recommended)

## **Handle Cancelled Orders Correctly**

Interpretation:

- Cancelled orders may still be useful (cancellation analytics)

- But they must not inflate revenue metrics incorrectly

Typical handling:

- keep cancelled orders
- ensure downstream analytics can exclude them for revenue if needed

## Payments Staging Transformations

### Payment Amount Validation Against Orders

Rule:

- `amount_paid` should match `orders.total_amount` for successful payments
- In practice, allow small rounding differences if needed, but rule says basic validation

Actions:

- Join payments with orders on `order_id`
- Create validation flag:
  - `is_amount_match = (amount_paid == total_amount)`

If mismatch:

- discard payment record OR mark as invalid based on strictness
- project requirement suggests discarding invalid records from staging

### Failed Payments Flag

- Create a boolean flag:
  - `is_failed_payment`

- Keep `payment_status`, but also store simplified analytic flag

## Remove Duplicate `payment_id`

- Deduplicate based on `payment_id`
- Prefer latest timestamp record if duplicates exist

## Order Items Staging Transformations

### Quantity Must Be Positive

Rule:

- `quantity > 0`

Discard if:

- 0 or negative

### Remove Null `product_id`

Reason:

- cannot join to product dimension
- cannot analyze product performance

### Derived Column Calculation

Create:

- `item_total = quantity * item_price - discount_amount`

Important considerations:

- Ensure `discount_amount` null becomes 0 before calculation

- Ensure item\_price is numeric and non-null (or discard)

## Inventory Staging Transformations

### Non-negative Stock

Rule:

- `quantity_available >= 0`

### Standardize location\_type

Allowed:

- Store
- Warehouse

Actions:

- trim and normalize case
- map values like "WH" → Warehouse if rule allows
- discard invalid if strict

### Remove Stale Records (If Necessary)

This depends on business definition of “stale”.

Typical rule:

- if last\_updated is older than a threshold (e.g., 30 days), treat it as stale

If applied:

- derive `record_age_days`
- filter out beyond threshold



# Clickstream Staging Transformations

Clickstream is often noisy and high-volume.

## Standardize Event Types

Allowed:

- view
- add\_to\_cart
- purchase
- search

Actions:

- normalize to lowercase
- discard unknown event types

## Derive event\_date for Partitioning

- `event_date = to_date(event_timestamp)`

Partitioning in staging:

- improves time-range filtering for behavior analytics

## Business Enrichment in Staging

Staging should become more analytics-friendly by bringing dimensional attributes closer.

### Enrichment Joins

#### Orders + Customers

Add:

- customer\_city (and possibly state)

Purpose:

- city-level sales reporting without needing extra joins later

### **Order Items + Products**

Add:

- category (and possibly brand, sub\_category)

Purpose:

- category-level revenue analytics

### **Payments + Orders**

Add order attributes to payments or align them in a unified dataset:

- order\_channel
- order\_date
- order\_status

Purpose:

- payment performance by channel/date/status

---

## **Staging Storage Strategy in Hive**

### **Managed vs External Tables in Staging**

Common choices:

- **Managed tables** in staging are acceptable because staging is re-creatable from raw.

- **External tables** are useful if you want independent data retention.

A safe enterprise approach:

- Raw = external (must be recoverable)
- Staging = managed (rebuildable)
- Analytics = managed or external depending on governance

## Partition Strategy for Staging

### **orders\_stg Partition by order\_date**

Reason:

- high-volume queries are almost always date filtered
- improves query speed and reduces scan costs

### **clickstream\_stg Partition by event\_date**

Reason:

- clickstream is time-series activity data
- typical queries are by day/week/month

## File Format and Compression in Staging

Recommended:

- Store staging in **Parquet** (columnar, fast)
  - Use **Snappy** compression (good performance and common standard)
-

# Analytics (Gold) Layer: Star Schema Design

The analytics layer is where business modeling happens.

## Analytics Layer Responsibilities

- Create Hive database: `{your_login_username}_analytics`
  - Read staging tables using Spark
  - Create star schema:
    - Fact table: `fact_sales_analytics`
    - Dimensions: customer, product, store, date
  - Ensure no duplicate records
  - Optimize for query performance (partitioning, parquet/orc, compression)
- 

## Star Schema Concepts

### Why Star Schema Instead of Normalized Tables

- Faster BI queries (fewer joins during reporting)
- Clear separation:
  - Facts = events/transactions
  - Dimensions = descriptive attributes

### Fact Table Definition: `fact_sales_analytics`

Derived from:

- `orders_stg`

- order\_items\_stg
- payments\_stg

Fact Table Columns

| Column Name    | Data Type | Description                                          |
|----------------|-----------|------------------------------------------------------|
| sales_key      | BIGINT    | Surrogate primary key (generated in analytics layer) |
| order_id       | STRING    | Business order identifier                            |
| customer_key   | INT       | Surrogate key from dim_customer                      |
| product_key    | INT       | Surrogate key from dim_product                       |
| store_key      | INT       | Surrogate key from dim_store (nullable for online)   |
| date_key       | INT       | Surrogate key from dim_date (YYYYMMDD format)        |
| order_channel  | STRING    | Online / Store                                       |
| order_status   | STRING    | Placed / Shipped / Delivered / Cancelled             |
| payment_status | STRING    | Success / Failed                                     |
| quantity       | INT       | Units purchased                                      |

|                 |           |                                                    |
|-----------------|-----------|----------------------------------------------------|
| gross_amount    | DOUBLE    | quantity × item_price                              |
| discount_amount | DOUBLE    | Discount applied                                   |
| net_amount      | DOUBLE    | gross_amount – discount_amount                     |
| payment_method  | STRING    | UPI / Card / NetBanking / Cash                     |
| load_timestamp  | TIMESTAMP | When this record was inserted into analytics layer |

## Duplicate Prevention in Fact

Define a natural grain:

- One row per:
  - (order\_id, product\_id, payment\_id) OR (order\_id, product\_id)  
depending on payment granularity

Common safe grain for sales analytics:

- (order\_id, product\_id) because each line item is one product in one order
- payment attributes are joined from payments (ensure single payment per order or resolve multiples)

---

## Dimension Tables Design

Dimensions should be deduplicated and enriched with derived fields.

### dim\_customer\_analytics

Source:

- customers\_stg

Derived fields required:

- customer\_key (surrogate)
- city, state
- customer\_segment (derived from spending)
- registration\_year

### **Customer Segment Derivation (Typical Approach)**

Use total spending from fact:

- Low: bottom tier
- Medium: mid tier
- High: top tier

Example segmentation table:

| <b>Segment</b> | <b>Rule Example<br/>(Illustrative)</b> |
|----------------|----------------------------------------|
| Low            | total_spend < 5,000                    |
| Medium         | 5,000 to 20,000                        |
| High           | > 20,000                               |

(Exact thresholds should be decided based on data distribution.)

### **dim\_product\_analytics**

Source:

- products\_stg (staging products table must exist conceptually)

Derived:

- product\_key
- category
- brand
- price\_band (Low/Medium/Premium)

### Price Band Derivation

Example:

| Band    | Rule Example |
|---------|--------------|
| Low     | price < 500  |
| Medium  | 500 to 2,000 |
| Premium | > 2,000      |

Thresholds should match business category (electronics vs grocery differ), but project expects a general banding.

### dim\_store\_analytics

Source:

- stores\_stg (staging stores table must exist conceptually)

Derived:

- store\_key
- region
- city
- store\_type (Tier1/Tier2 derived)

### Tier Derivation Idea

Tier could be derived from:



- store city classification (metro vs non-metro)
- store revenue tiers once fact is built

Example:

| Tier  | Rule Example                      |
|-------|-----------------------------------|
| Tier1 | metro cities / top revenue stores |
| Tier2 | others                            |

## dim\_date\_analytics

Usually generated rather than sourced:

- date\_key (YYYYMMDD)
- date, day, month, quarter, year
- week\_of\_year, is\_weekend

This dimension makes time-based reporting faster and consistent.

---

# Analytics Transformations Required in Spark

## Join Staging Tables to Build Fact

Typical build steps:

1. Start with `orders_stg`
2. Join `order_items_stg` on `order_id` to get product lines
3. Join `payments_stg` on `order_id` or `payment_id` depending on design
4. Compute:

- gross\_amount
  - net\_amount
5. Join dimensions to get surrogate keys:
- customer\_key from dim\_customer
  - product\_key from dim\_product
  - store\_key from dim\_store
  - date\_key from dim\_date
6. Generate `sales_key` (surrogate PK)
7. Deduplicate based on chosen grain
8. Load into `{username}_analytics.fact_sales_analytics`

## Partition Fact Table by Year/Month

Partition keys derived from order\_date:

- year = year(order\_date)
- month = month(order\_date)

Reason:

- large table
- frequent time filtering
- improves pruning and query performance

## Hive Table Creation Strategy for Analytics

### File Formats

- Fact table:
  - **Parquet** (required)
  - **Snappy** compression
- Dimension tables:
  - ORC or Parquet (both are acceptable)
  - choose based on standardization and performance expectations

## Why Columnar Storage Matters

- Analytics queries scan a few columns but many rows
- Columnar formats avoid reading unnecessary data
- Compression is more effective column-wise

## Business Analytics Outputs to Generate

These are typical business questions answered from the analytics layer.

### Top Analysis

#### Top 10 Products by Revenue

Logic:

- sum net\_amount grouped by product\_key (or product\_id)
- order by revenue desc
- take top 10

#### Top 10 Customers by Spending

Logic:

- sum net\_amount grouped by customer\_key

- order desc
- top 10

## **Top Performing Stores**

Logic:

- sum net\_amount grouped by store\_key
- exclude null store\_key if you only want physical stores
- rank by revenue

## **Customer Insights**

### **Repeat Customer Percentage**

Definition idea:

- customers with more than 1 distinct order / total customers with orders

### **Customer Lifetime Value (CLV)**

Common simplified CLV for project:

- total net\_amount per customer over available data window

### **Average Order Value (AOV)**

- total net\_amount / total distinct orders

### **Monthly Active Customers**

- count distinct customers per month based on order\_date

## **Store Performance**

### **Revenue per Store**

- sum net\_amount grouped by store\_key

### Revenue per Region

- join store dimension to region
- sum net\_amount by region

### Store-wise Conversion (If Clickstream is Added)

Requires aligning:

- clickstream sessions / events to purchases
- conversion rate = purchases / visits or purchases / add\_to\_cart depending on definition

## Deliverables: Output Files in HDFS

For assessment, create output files for each business problem and store them in:

- HDFS folder: `business output`
- Required folder name: `{your_login_username}_analytics_output/`

### Recommended Output Organization

- `{username}_analytics_output/top_products_by_revenue/`
- `{username}_analytics_output/top_customers_by_spending/`
- `{username}_analytics_output/top_stores/`
- `{username}_analytics_output/repeat_customer_percent/`
- `{username}_analytics_output/clv/`
- `{username}_analytics_output/aov/`

- `{username}_analytics_output/monthly_active_customers/`
- `{username}_analytics_output/revenue_per_store/`
- `{username}_analytics_output/revenue_per_region/`

Store outputs in Parquet/CSV as required by evaluation, but Parquet is preferred for scale.

---

## Conclusion

### Key Insights

- A modern retail data platform typically follows layered architecture: **Raw** → **Staging** → **Analytics**, each serving a distinct purpose.
- The **raw layer** must be immutable and recoverable, so using **external Hive tables** and stable HDFS storage is crucial.
- The **staging layer** is where data becomes reliable: duplicates, invalid values, type mismatches, and corrupt records are removed using Spark, and jobs must not fail due to bad records.
- The **analytics layer** should follow a **star schema** to support fast reporting: a single fact table for sales and multiple dimensions for descriptive attributes.
- Performance depends heavily on **partitioning strategy** (order\_date, event\_date; year/month for fact), and on using **columnar formats** like Parquet with **Snappy compression**.

### Key terms

- **Raw layer:** Source-aligned storage that preserves data as received with minimal changes for audit and replay.
- **Staging layer:** Cleaned and standardized data layer used as a trusted base for downstream modeling.

- **Analytics (Gold) layer:** Business-modeled layer optimized for reporting, often using star schema.
- **External Hive table:** Hive table where data remains in HDFS even if table metadata is dropped.
- **Managed Hive table:** Hive table where dropping the table deletes both metadata and underlying data.
- **Partitioning:** Splitting table data into directories by a key (like date) to reduce scan and speed queries.
- **Bucketing:** Hash-based distribution into fixed buckets to speed joins/aggregations when used correctly.
- **Star schema:** Data model with a central fact table connected to multiple dimension tables for analytics.
- **Surrogate key:** System-generated key (not from source) used for stable dimension joins.
- **Fact table grain:** The exact meaning of one row in a fact table (for example, one row per order line item).
- **Columnar storage (Parquet/ORC):** File formats optimized for reading selected columns efficiently in analytics workloads.